

ConvFill: Model Collaboration for Responsive Conversational Voice Agents

Vidya Srinivas*, Zachary Englhardt*, Maximus Powers, Shwetak Patel, Vikram Iyer

Paul G. Allen School of Computer Science & Engineering

{vysri, zacharye} @cs.washington.edu

Abstract

Deploying conversational voice agents with large language models faces a critical challenge: cloud-based foundation models provide deep reasoning and domain knowledge but introduce latency that disrupts natural conversation, while on-device models respond immediately but lack sophistication. We propose *conversational infill*, a task where a lightweight on-device model generates contextually appropriate dialogue while seamlessly incorporating streaming knowledge from a powerful backend model. This approach decouples response latency from model capability, enabling systems that feel responsive while accessing the full power of large-scale models. We present **ConvFill**, a 360M parameter model trained on **synthetic multi-domain conversations**. Evaluation across multiple backend models shows that conversational infill can be successfully learned, with ConvFill achieving **accuracy improvements of 36-42%** over standalone small models of the same size while consistently retaining **sub-200ms** response latencies. Our results demonstrate the promise of this approach for on-device building conversational agents that are both immediately responsive and knowledgeable.

1 Introduction

With the growing use of flagship large language models (LLMs) and agentic workflows across diverse domains (Laranjo et al., 2018; Casheekar et al., 2024), there is also an increasing adoption of conversational systems in areas such as digital assistance, education, customer service, coaching, and beyond. However, deploying these systems in real-time, interactive voice settings presents unique challenges. On-device models can respond immediately but lack the reasoning depth of large cloud models. Cloud-based foundation models can introduce latency prohibitive to natural conversation.

*equal contribution

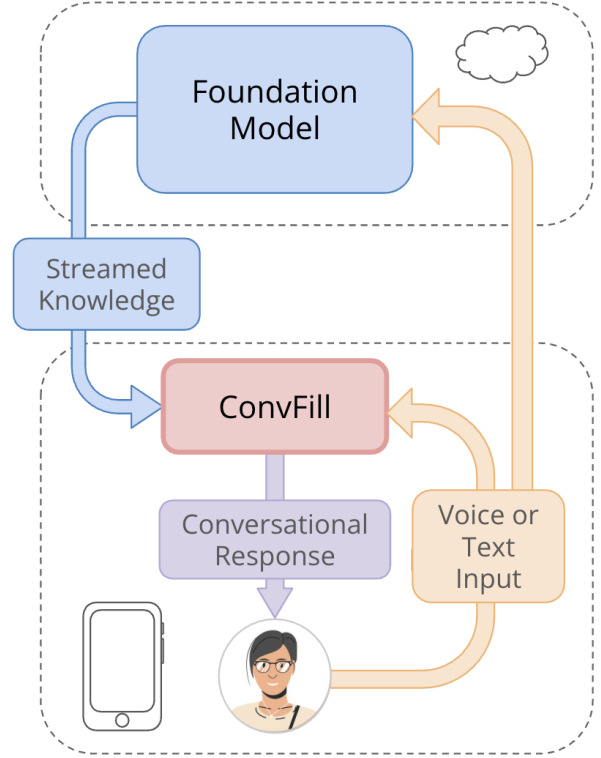


Figure 1: **Conversational Infill**. ConvFill operates on the turn level while the backend model operates on the conversation level. The backend model outputs silence or a knowledge chunk (blue) and ConvFill incorporates these chunks along with conversational infill to generate the response to the user (orange).

Existing approaches to real-time voice assistants fall broadly into one of two categories. **Cascaded systems** combine separate automatic speech recognition (ASR), LLM, and text-to-speech (TTS) components. While this modularity allows easy integration of new models or domain-specific LLMs, these systems suffer from compounding latencies. Even when TTS latency is ignored, the time-to-first-response depends entirely on the LLM’s generation speed. For large cloud based models, especially reasoning models with rich domain knowledge, this delay can introduce awkward pauses that break conversational flow. **End-to-end speech models** (Dé-

fosse et al., 2024; Iribe et al., 2024; Gao et al., 2025) generate audio outputs directly, achieving lower latency by producing speech without requiring dedicated ASR and TTS steps. However, these models face significant limitations. They generally tie the **entire system to a single LLM backbone**, sacrificing the flexibility to swap in more powerful or specialized foundation models. The need for real-time performance also constrains model size, **limiting their reasoning and domain knowledge** compared to state-of-the-art text-based LLMs.

This raises a central research question: *How can we design a conversational voice interface that achieves the low latency and responsiveness of on-device small models while maintaining access to the sophisticated reasoning and domain knowledge of large cloud-based foundation models?*

We introduce a hybrid conversational architecture that addresses this gap through model collaboration. ConvFill implements what we formally define as the *conversational infill* task: generating natural, contextually appropriate dialogue that maintains conversational flow while a high-capacity model processes in the background. A lightweight **on-device model immediately begins responding conversationally to the user input without relying on generic infill phrases**, maintaining natural dialogue flow with minimal time-to-first-response. Simultaneously, **a high-capacity cloud-based foundation model processes the conversation in the background**, enabling more sophisticated reasoning and in-depth factual information. The infill model seamlessly incorporates outputs from the cloud model as they become available, hiding latency through natural conversational behavior. This modular design enables systems that combine the responsiveness benefits of on-device deployment with the powerful capabilities of cloud foundation models, while remaining flexible enough to integrate different backend LLMs across domains and use cases.

2 Fillers in Conversation

While conversational fillers may bring to mind simple utterances like “um”, “uh”, or “hmm”, recent work has expanded this notion to encompass richer, **sentence-level responses**. (Gao et al., 2025) propose phased responses that provide early acknowledgment and context while processing continues—for instance, confirming receipt of a query in tool-use scenarios before the full answer is ready.

This approach demonstrates that effective fillers can span multiple sentences and add substantive value beyond simply occupying time. For example, when a user asks “What’s the weather like in Tokyo next week?”, rather than responding with “um” or silence, a system might immediately say “Let me check the forecast for Tokyo...”, acknowledging the request and setting expectations while the system retrieves data.

Building on this expanded view, (Mahmood et al., 2025) categorize fillers into functional types (small talk, continuing, introducing, closing) to implement state-machine based conversation management. Meanwhile, (Lew et al., 2018) empirically demonstrate that conversational contingency, or how well responses fit the conversation flow, is preferred over simply minimizing latency. Their findings show that **users rank contextually appropriate responses higher than faster but generic ones**.

We adopt this broader definition of conversational fillers as **context-sensitive, sentence-level responses** that maintain conversational flow. Like (Gao et al., 2025), we view fillers not as simple placeholder words but as intelligent responses that can span multiple sentences, acknowledge user input, and provide incremental value while more complex processing occurs in the background.

3 Problem Formulation

We propose a novel task called *conversational infill*, in which a small, on-device conversational *infill* model accepts **external textual knowledge chunks, potentially delayed in time**, from a large, off-device *backend* language model **in a streaming manner**, to enhance its responses during inference time. We define textual *entailment* as it is defined in (Williams et al., 2017). Particularly, **entailment** means that a given **premise guarantees the truth of a corresponding hypothesis**. This is in contrast to *contradiction* which means that a premise guarantees the falsity of a hypothesis, and *neutral* which means that the a premise neither guarantees the truth nor the falsity of the hypothesis. In the following sections, we use the notation $a \rightarrow b$ to indicate that *a* entails *b*.

In the conversational infill task, the small infill model, accepts the contents of a single turn as input (i.e., the most recent user utterance and external knowledge chunks). In contrast, the **backend model, which provides knowledge, sees the entire conversation history** (i.e., all previous exchanges between the user and the infill model). This enables

the infill model to be fast and responsive, handling a simpler task, while the burden of **coherent information generation and context management is dedicated to the larger backend model**. An illustration of this concept is provided in Figure 1.

We provide a formal definition of the conversation infill task as follows. In subsequent text, we use t to refer to the time in seconds that has passed from the start of each conversational turn for the on-device model, where $i \in t$. We refer to the external knowledge (with respect to the infill model) as E , where E is a list of consisting of **streamed knowledge chunks** e_i . Each e_i can either be a **knowledge chunk** from the backend model, which we refer to as $l_i \in L$ or a **special silence token**, $s_i \in S$. A silence token is generated by our system when a duration of d seconds has passed since the receipt of the latest knowledge chunk l_{i-d} .

At each turn, at time i , the **infill model** receives external **input** of the form of a list $(e_1, \dots, e_i)$, as well as $(c_1, \dots, c_{i-1})$, a list of its own previous responses for the current turn. It uses these inputs to generate a **conversational phrase** c_i , meaning that the conversational phrase that starts generation at time i is generated from the streaming knowledge chunks and previous conversational phrases seen at time i . n refers to the total number of such conversational phrases, per turn. In our task, we require that $\text{len}(S) + \text{len}(L) = \text{len}(C) = n$.

Particularly, in our task formulation, $e_i \rightarrow c_i$ if $e_i \in L$. This means that conversational phrases generated from backend knowledge must only add **intelligent fillers**, but no other information. Conversational phrases derived from silence, i.e. $e_i \in S$, are **conversational fillers** and are generated by the infill model alone without relying on external knowledge. There is relationship between s_i and c_i for any i . The infill model generates its conversational phrases without reliance on the backend model in these cases.

4 Infill Model Training and Inference

In the following subsections we provide a description of our model training, dataset generation, and inference pipelines.

4.1 Dataset Generation

While it may initially seem that transcribing existing conversational datasets may be a good fit for this task, large datasets such as **CANDOR** (Reece et al., 2023) consist of casual conversation. Al-

though fillers have been shown to be helpful in masking latency (Maslych et al., 2025), **substantial utilization of simple filler words** by conversational agents has been shown to **negatively impact** user perception (Jeong et al., 2019). In order to develop a training corpus of goal-directed conversations without excessive use of filler words, we opt to generate a synthetic dataset.

We generate a **synthetic conversation dataset using GPT-4o**. The dataset is comprised of different conversations from the domains of advice, assistant queries, education, event planning, customer service, medical conversation, with **1000 per domain**. Conversations vary from **8-12 turns**, and emulate spoken dialogue between two people. For the topics of medical, customer service, and education, we first generate a list of **1000 domain-specific personas**. We use these personas to increase variety in generated conversations and to generate domain specific conversations that cover reasonable queries from an everyday user. We find that personas seeded from **PersonaHub** (Ge et al., 2025) are too specific for this use case (e.g. "A florist and bird lover based in Herne Bay, Kent."), leading to situations in which niche details from the persona derail the conversation. We then use the generated personas to seed the model role in conversation generation. For advice, assistant, and planning topics, we generate 1000 domain-specific subtopics and use these to seed conversation generation.

Each conversation is generated as a JSON and contains the fields **user**, **responder**, and **responder_thoughts**. **user** contains the user utterance for that turn, **responder** contains a list of standalone sentences said by the model, and **responder_thoughts** concise thoughts or silence that correspond to each of the elements of **responder**. In the next section, we explain how this dataset is used for training.

Although this generation task may seem straightforward, it is potentially quite error prone. We initially attempted a simulated conversation using multiple inference turns, but we find these **simulated conversations performed poorly due to lost contingency**. In other words, the conversations failed to progress to logical next steps beyond the initial premise towards a goal-oriented outcome.

Within each turn, it is critical that the streamed knowledge chunks contain the necessary information to generate the conversational response. To prevent this semantic mismatch from appearing in the generated dataset, we utilize a publicly avail-



Figure 2: **Infill generation format.** During training, ConvFill sees the user utterance, previous streamed knowledge chunks, and previous conversational phrases in an interleaved manner. ConvFill is trained to predict the last red utterance—its own last conversational phrase conditioned on the last external knowledge chunk (blue) and the phrase history.

able (noa) version of DeBERTaV3 (He et al., 2023) finetuned on the MNLI dataset (Williams et al., 2017) to verify each streamed knowledge chunk entails the corresponding conversational phrase.

In this work, we generate 5997 conversations spanning the domains of advice, assistant queries, education, event planning, customer service and medical conversation. We define a conversational turn as the collection of one user utterance, a list of streamed knowledge chunks, and a list of their corresponding conversational phrases (reference Section 3 for definitions). Altogether, from these conversations, we derive a total of 46151 user-infill turns, which we use for model training.

4.2 Training Pipeline

We use SmoLM2-360M-Instruct (Allal et al., 2024) as the base of our infill model (hereafter referred to as ConvFill) and finetune it to perform the conversational infill task. We choose this model because it is small and enables quick inference, even on lightweight devices. We split the dataset in Section 4.1 into individual turns, each of which contains the user utterance, a list of streamed knowledge chunks, and a list of conversational phrases. SmoLM2 already supports role tags for user and assistant roles, which we use for user turn and conversational phrases, respectively; however to introduce conditional generation on a large model stream, we introduce a new tag, `knowledge`. The content for this knowledge stream is generated from our system and either contains a large model utterance or `<|sil|>`, a silence token. For a full description of the system and problem formulation, refer to Section 3.

We then train ConvFill with a prompt format as specified in Figure 2. In this format, ConvFill only sees the user utterance for the current turn as well

as previous interleaved external knowledge chunks and previous conversational phrases for the current turn only. At all times, the small model is asked to predict only the conversational last utterance for that turn. ConvFill is trained on an L40 GPU for five epochs in total, with a learning rate of $1e-5$, 500 warmup steps, and a cosine learning rate scheduler. We add the special token, `<|sil|>`, to the tokenizer and `resize embeddings` as necessary. Further specifications can be found in our code, which will be released at a later time.

4.3 Inference Pipeline

During training, we save the tokenizer, and restore it for inference. Two parallel threads run during inference, one with a large model and one with ConvFill. These threads share a *streamed knowledge queue*. When a user utterance is passed to our system, we send the query in parallel to ConvFill and the large model. A dialogue manager for the large model appends the user utterance to conversational history. In the rest of the turn, for response generation, our system passes a silence token to ConvFill every 1 second ($d = 1$, refer to Section 3 for definition of d) if there is not already a response in the streamed knowledge queue. Inference is performed in a streaming manner as described in Section 3. An example of ConvFill generation for a real scenario can be found in Figure 3.

In our main experiments, we use Gemini 2.5 Pro as the large model for streaming knowledge chunks. However, in practice, any model may be used as the backend given that it responds in phrases that can be passed to ConvFill. For API-based foundation models specifically, we prompt models to respond concisely to avoid the usual, sprawling format of normal back and forth generation and ensure that the models act as knowledge sources rather than conversationalist frontends.

5 Evaluation

In this section, we provide three evaluations: question-answering (QA) accuracy, time-to-first-token (TTFT) latency, and turn-level entailment, each of which highlights a critical dimension of the conversational infill task. Particularly, QA accuracy measures if the ConvFill model is able to achieve a superior accuracy to its non-ConvFill base model counterpart within the same number of parameters. TTFT latency measures each model’s response time, a crucial metric for real-time conver-

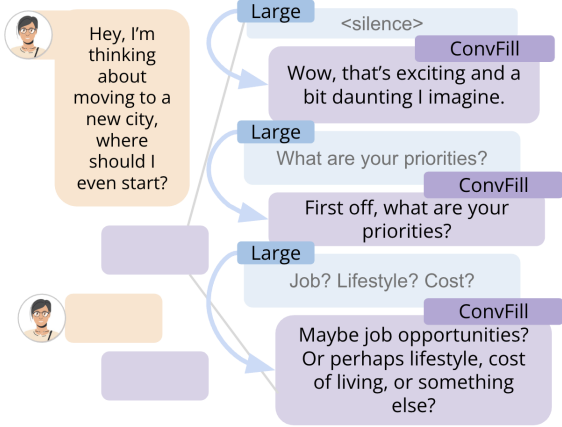


Figure 3: **Conversational infill inference example.** The user asks about moving to a new city. ConvFill generates a response in an interleaved and streaming manner, in each conversational phrase (purple), referencing an external knowledge chunk (blue).

sational interfaces. *Turn-level entailment* measures if the ConvFill model is able to preserve and use information from a large knowledge model in a manner that is consistent with this external knowledge stream. These aspects are critical to ensure that the ConvFill model is not only useful, but also low-latency and grounded.

5.1 Methods

We evaluate the ConvFill model on a randomly selected subset of the **NaturalQuestions** dataset (Kwiatkowski et al., 2019), a challenging open-domain question answering benchmark. We compare the performance of the ConvFill model with three different backend models (GPT-5, Claude Sonnet 4.5, Gemini-2.5-Pro) against the performance of the backend models themselves for QA accuracy and TTFT latency. We also include analysis of multiple similarly-sized open small language models to contextualize performance on these two metrics.

For the **entailment portion** of the evaluation, we utilize a publicly available (noa) version of **DeBERTaV3** (He et al., 2023) finetuned on the **MNLI dataset** (Williams et al., 2017) to verify, per turn, whether the combination of the current streamed knowledge chunk and previous in-turn conversational phrases entail the corresponding conversational phrase from ConvFill. Particularly, we verify that within the generation format provided in Figure 2, ConvFill is able to draw on its own previous utterances as well as the most recent external knowledge that it is provided to generate a

Table 1: TTFT (Time To First Token) is measured in seconds. Param indicates on-device parameter count for ConvFill configurations. ConvFill consistently achieves 4-67× faster response times compared to backend models, and is in-line with base model and comparable small on-device-class model latencies.

Model	Param	TTFT± std (s)
GPT-5	N/A	0.74 ± 3.04
ConvFill + GPT-5	360M	0.16 ± 0.04
Claude Sonnet 4.5	N/A	2.16 ± 0.99
ConvFill + Claude 4.5	360M	0.17 ± 0.04
Gemini-2.5-Pro	N/A	10.9 ± 2.74
ConvFill + Gemini	360M	0.17 ± 0.04
SmolLM2-360M (Base)	360M	0.16 ± 0.04
SmolLM2-1.7B	1.7B	0.17 ± 0.18
Qwen3-0.6B	600M	0.15 ± 0.05
Qwen1.7B	1.7B	0.20 ± 0.24
Llama3.2-1B	1B	0.18 ± 0.13
Gemma3-270M	270M	0.14 ± 0.07
Gemma3-1B	1B	0.22 ± 0.08

grounded response.

For each evaluation, we randomly sample 250 questions. The ConvFill system operates in a multi-turn streaming pipeline as described in Section 4.3 and illustrated in Figure 3. The silence token `<sil>` is provided as input when no knowledge stream is available. This means that the ConvFill system is operated in these evaluation as it would be during streaming inference. All other non-ConvFill models are directly prompted with the dataset questions with no additional token insertions. Local models are all quantized to 16-bit floating point values (fp16) and inference is performed using Ollama (Oll, 2025) on a Apple MacBook Pro (M2 Pro, 19 core GPU, 16 GB).

5.2 Results

TTFT Latency. TTFT results shown in Table 1 demonstrate that ConvFill achieves consistently low TTFT latencies across all backend configurations. While standalone Claude Sonnet 4.5 and Gemini-2.5-Pro exhibit mean TTFTs of 2.16s and 10.9s respectively, ConvFill reduces these to approximately 0.17s in both cases. Even GPT-5, which has a relatively fast baseline TTFT of 0.74s, is reduced to 0.16s when paired with ConvFill. Notably, all ConvFill configurations achieve TTFTs under 200ms with low variance (± 40 ms).

QA Accuracy. QA accuracy results shown in Table 2 reveal important trade-offs. Standalone

Table 2: Performance comparison on Natural Questions (NQ) dataset (250 samples). ConvFill consistently achieves increased performance relative to the base model, but fails to match the full performance of the backend model alone.

Model	NQ Accuracy
GPT-5	0.78
ConvFill + GPT-5	0.46
Claude Sonnet 4.5	0.80
ConvFill + Claude 4.5	0.52
Gemini-2.5-Pro	0.69
ConvFill + Gemini	0.49
SmolLM2-360M (Base)	0.10
SmolLM2-1.7B	0.26
Qwen3-0.6B	0.12
Qwen1.7B	0.28
Llama3.2-1B	0.34
Gemma3-270M	0.06
Gemma3-1B	0.19

SmolLM2-360M achieves only 10% accuracy on NaturalQuestions, while ConvFill configurations achieve 46% to 52% accuracy depending on the backend model. This substantial improvement demonstrates that the local model successfully incorporates knowledge from the backend stream. However, ConvFill does not fully recover the accuracy of the backend models themselves 69%-80%, indicating limitations in how effectively the 360M parameter model can leverage streaming knowledge signals.

Turn-Level Entailment. ConvFill entailment rates only remain at 28.1%, 34.1%, and 35.5% for backend models of Claude Sonnet 4.5, GPT-5, and Gemini 2.5 Pro, respectively, as shown in Table 3. This demonstrates that ConvFill responds only relying on its context, not changing meaning, in about 1/3 of cases. Percentages of neutral ratings for those models are entailment rates only remain at 64.2%, 59.2%, and 59.9%, respectively. At first glance, this may seem like ConvFill fails to remain grounded. However, upon closer manual inspection, we find that many examples that are marked as neutral remain realistically entailed. Contradiction rates remain low across all backend models, at 5%-7%.

6 Discussion

TTFT Latency. ConvFill presents a promising paradigm for hybrid inference: users experience the low-latency responsiveness of a local model while still benefiting from the knowledge and reasoning

Table 3: Natural Language Inference analysis of ConvFill responses compared to backend models (measured on response pairs). Percentages indicate the proportion of entailment (E), neutral (N), and contradiction (C) relationships.

Backend Model	Pairs	E (%)	N (%)	C (%)
GPT-5	414	34.1	59.9	6.0
Claude Sonnet 4.5	495	28.1	64.2	7.7
Gemini-2.5-Pro	245	35.5	59.2	5.3

capabilities of large-scale backend models. Particularly, regardless of backend model, ConvFill consistently achieves TTFTs under 200 ms, demonstrating its ability to be used in cascaded conversational systems. Furthermore, the benefit of such a system may be higher for thinking and agentic models and that respond with significantly higher latency. This approach offers a powerful technique for deploying high-quality conversational AI systems where both immediate feedback is critical.

QA Accuracy. The ConvFill system significantly boosts the accuracy of the baseline SmolLM2-360M model by 36-42%, demonstrating the potential of using the ConvFill approach to share the knowledge of a large cloud-based model with a small on-device model. However, the fact that the ConvFill system still does not match the performance of the backend knowledge model reveals a gap in the small model’s ability to effectively incorporate all external knowledge. This gap represents a key area for future improvement, potentially through larger on-device models, improved training objectives, or better knowledge integration mechanisms.

Turn-Level Entailment. The entailment analysis reveals nuances in how conversational language differs from strict logical inference. Responses classified as entailment (28.1-35.5% across backend models) generally demonstrate correct information preservation, indicating that ConvFill successfully incorporates external knowledge without distortion in these cases. The low contradiction rates (5.3-7.7%) are encouraging, as these represent cases where ConvFill introduces factually incorrect or conflicting information—the most problematic failure mode. However, the majority of responses (59.2-64.2%) are classified as neutral, warranting closer examination. Manual inspection reveals that many neutral classifications arise from natural conversational embellishments that, while technically changing the semantic content,

could be considered appropriate and truthful. Consider the following example: when the backend provides: *"Jack Nicklaus is indeed one of the most successful Masters winners. His wins were between 1963 and 1986,"* ConvFill generates: *"He had a string of amazing wins, from 1963 to 1986, which are impressive indeed."* The NLI model classifies this as neutral because ConvFill introduces evaluative language ("amazing," "impressive") not present in the original knowledge chunk. Whether such additions constitute acceptable paraphrasing or meaning-changing embellishment is inherently context-dependent. **In formal information retrieval, adding subjective adjectives would be inappropriate. In conversational settings, however, these enhancements can contribute to natural dialogue flow and engagement without compromising factual accuracy.** Assessing this is inherently more fluid and interpretive than the strict logical relationships captured by textual entailment frameworks. These results suggest that while NLI-based evaluation provides useful bounds such as identifying clear entailments and obvious contradictions, the neutral category currently requires human judgment to distinguish between acceptable conversational variation and problematic semantic drift. Future work might directly leverage novel evaluation frameworks better aligned with conversational norms rather than strict logical inference.

7 Related Work

Recent work has explored dividing conversational and reasoning responsibilities across multiple models to balance responsiveness and intelligence. Mini-Omni-Reasoner (Xie et al., 2025) introduces token-level thinking-in-speaking, in which reasoning and speech generation occur concurrently. (Christakopoulou et al., 2024) introduces a Talker-Reasoner framework in which a large model shares reasoning and state with a small model that collects user context. SplitReason (Akshauri et al., 2025) explores reasoning offloading, where lightweight reasoning models defer complex reasoning to powerful backend models. Qwen2.5-Omni (Xu et al., 2025) presents a Thinker-Talker architecture where the Thinker generates text and the Talker takes high level representations and generates streaming speech tokens.

Another line of research focuses on lowering the latency of model responses to support real-time interaction. Speculative and online speculative decod-

ing (Leviathan et al., 2023; Liu et al., 2023) allow faster preliminary generation that is later verified or refined by a larger model, improving effective decoding speed. (Li and Grover, 2025) extends this to cascaded speech models by generating candidate responses while the user is still speaking, enabling the system to begin TTS processing earlier.

8 Conclusion

We introduce **conversational infill**, a novel task in which a small model generates natural dialogue while incorporating streaming knowledge from a backend model to maintain conversational flow and factual grounding. Our proof-of-concept implementation, ConvFill, demonstrates the feasibility of this task: a 360M parameter on-device model achieves sub-200ms response latencies comparable to standalone small models while substantially improving accuracy (10% → 46-52% on NaturalQuestions) by successfully incorporating streamed backend model knowledge from various backend foundational language models. Although promising, the gap between ConvFill and backend model performance (69-80%), along with moderate entailment rates (28-35%), reveals room for future improvements.

References

- MoritzLaurer/DeBERTa-v3-base-mnli · Hugging Face. 2025. Ollama. Original-date: 2023-06-26T19:39:32Z.
- Yash Akshauri, Anthony Fei, Chi-Chih Chang, Ahmed F AbouElhamayed, Yueying Li, and Mohamed S Abdelfattah. 2025. Splitreason: Learning to offload reasoning. *arXiv preprint arXiv:2504.16379*.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. 2024. Smollm - blazingly fast and remarkably powerful.
- Avyay Casheekar, Archit Lahiri, Kanishk Rath, Kaushik Sanjay Prabhakar, and Kathiravan Srinivasan. 2024. A contemporary review on chatbots, ai-powered virtual conversational agents, chatgpt: Applications, open challenges and future research directions. *Computer Science Review*, 52:100632.
- Konstantina Christakopoulou, Shibl Mourad, and Maja Matarić. 2024. Agents thinking fast and slow: A talker-reasoner architecture. *arXiv preprint arXiv:2410.08328*.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. 2024. *Moshi: a speech-text foundation model for real-time dialogue. Preprint*, arXiv:2410.00037.

- Heting Gao, Hang Shao, Xiong Wang, Chaofan Qiu, Yunhang Shen, Siqi Cai, Yuchen Shi, Zihan Xu, Zuwei Long, Yike Zhang, Shaoqi Dong, Chaoyou Fu, Ke Li, Long Ma, and Xing Sun. 2025. [Lucy: Linguistic understanding and control yielding early stage of her](#). *Preprint*, arXiv:2501.16327.
- Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. 2025. [Scaling synthetic data creation with 1,000,000,000 personas](#). *Preprint*, arXiv:2406.20094.
- Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. [DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing](#). *arXiv preprint*. ArXiv:2111.09543.
- Brendan Iribe, Ankit Kumar, and The Sesame Team. 2024. [Crossing the uncanny valley of voice](#). Blog post.
- Yuin Jeong, Juho Lee, and Younah Kang. 2019. [Exploring effects of conversational fillers on user perception of conversational agents](#). In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI EA '19, page 1–6, New York, NY, USA. Association for Computing Machinery.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. [Natural questions: A benchmark for question answering research](#). *Transactions of the Association for Computational Linguistics*, 7:452–466.
- Liliana Laranjo, Adam G Dunn, Huong Ly Tong, Ahmet Baki Kocaballi, Jessica Chen, Rabia Bashir, Didi Surian, Blanca Gallego, Farah Magrabi, Annie YS Lau, and 1 others. 2018. Conversational agents in healthcare: a systematic review. *Journal of the American Medical Informatics Association*, 25(9):1248–1258.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. [Fast inference from transformers via speculative decoding](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR.
- Zijian Lew, Joseph B Walther, Augustine Pang, and Wonsun Shin. 2018. Interactivity in online chat: Conversational contingency and response latency in computer-mediated communication. *Journal of Computer-Mediated Communication*, 23(4):201–221.
- Shufan Li and Aditya Grover. 2025. Predgen: Accelerated inference of large language models through input-time speculation for real-time speech interaction. *arXiv preprint arXiv:2506.15556*.
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2023. Online speculative decoding. *arXiv preprint arXiv:2310.07177*.
- Amama Mahmood, Junxiang Wang, Bingsheng Yao, Dakuo Wang, and Chien-Ming Huang. 2025. User interaction patterns and breakdowns in conversing with llm-powered voice assistants. *International Journal of Human-Computer Studies*, 195:103406.
- Mykola Maslych, Mohammadreza Katebi, Christopher Lee, Yahya Hmaiti, Amirpouya Ghasemaghahi, Christian Pumarada, Janneese Palmer, Esteban Segarra Martinez, Marco Emporio, Warren Snipes, Ryan P. McMahan, and Joseph J. LaViola Jr. 2025. [Mitigating response delays in free-form conversations with llm-powered intelligent virtual agents](#). In *Proceedings of the 7th ACM Conference on Conversational User Interfaces*, CUI '25, New York, NY, USA. Association for Computing Machinery.
- Andrew Reece, Gus Cooney, Peter Bull, Christine Chung, Bryn Dawson, Casey Fitzpatrick, Tamara Glazer, Dean Knox, Alex Liebscher, and Sebastian Marin. 2023. [The candor corpus: Insights from a large multimodal dataset of naturalistic conversation](#). *Science Advances*, 9(13).
- Adina Williams, Nikita Nangia, and Samuel R Bowman. 2017. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426*.
- Zhifei Xie, Ziyang Ma, Zihang Liu, Kaiyu Pang, Hongyu Li, Jialin Zhang, Yue Liao, Deheng Ye, Chunyan Miao, and Shuicheng Yan. 2025. Mini-omni-reasoner: Token-level thinking-in-speaking in large speech models. *arXiv preprint arXiv:2508.15827*.
- Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, and 1 others. 2025. Qwen2.5-omni technical report. *arXiv preprint arXiv:2503.20215*.